

Trustworthy Intent in Speech (TIS) Codebase Documentation

version

2024, Constantina Maltezou-Papastyliou

November 24, 2024

Contents

Trustworthy Intent in Speech (TIS) Codebase API Documentation	1
Codebase scripts	1
Modules Overview	1
main Script	1
main.py	1
Custom Packages	2
Machine Learning	2
random_forest_module.py	2
regression_ML_module.py	2
Stats	3
normality_tests_module.py	3
descriptives_module.py	3
regression_stats_module.py	4
Preprocessing	4
feature_encoding_module.py	4
dataset_stratification_module.py	5
Misc	5
visual_formatting_module.py	5
data_visualisation_module.py	6
helpers_module.py	7
License	8
Index	9
Python Module Index	11

Trustworthy Intent in Speech (TIS) Codebase API Documentation

Welcome to the **API Documentation of the Python codebase used for TIS!** The Python codebase along with the speech corpora dataset and the rest of the material of this project are made available as open-access to facilitate research and development in the fields of voice perception, speech processing, and related areas.

The scripts in this codebase are meant to describe and evaluate the speech dataset, particularly the acoustic features that differentiate natural speech (i.e. neutral or no intent) from speech intended to convey trustworthiness. The dataset includes data from 1,152 speech recordings by 96 untrained speakers (i.e. non-actors), covering a diverse range of demographics (age, sex, ethnicity) and speech intent - natural speech vs. deliberate attempts to convey trustworthiness.

The codebase provided here is designed to:

- **Preprocess** the CSV data relating to extracted acoustic features and speaker demographics.
- **Classify** speech based on its intent (trustworthy vs. neutral).
- **Validate** the dataset's effectiveness in categorising speech according to speaker intent and demographics.
- **Visualise** the relationship between speaker demographics, acoustic features and trustworthy intent classifications.

Codebase development and Licensing:

- This codebase supports the analyses for the upcoming publication "Human voices communicating trustworthy intent: A demographically diverse speech audio dataset" by Constantina Maltezou-Papastyliaou, Reinhold Scherer and Silke Paulmann.
- Developed by: Constantina Maltezou-Papastyliaou.
- Licensed under the wider project license: CC BY 4.0 International License.

For detailed instructions on how to set up the environment, run the analysis scripts, and attribute the code, refer to the **README.md** file.

Codebase scripts

Modules Overview

main Script

main.py

This is the main script to perform the speech dataset analyses described below.

1. Pre-process the data for analysis:

1.1. Encode categorical variables into numerical format. 1.2. Stratify the dataset by speaker demographics (e.g., ethnicity, age group).

2. Calculate descriptive statistics for speaker demographics and acoustic features.

3. Evaluate the distribution of the data.

4. Assess the importance of acoustic features, and reliability of the dataset using:

4.1. Classification methods (i.e., cross-validated Random Forest and Logistic Regression from the sklearn package). 4.2. Feature importance (sklearn and statsmodels packages).

5. Codebase development and Licensing:

5.1. This codebase supports the analyses for the upcoming publication “Human voices communicating trustworthy intent: A demographically diverse speech audio dataset” by Constantina Maltezou-Papastilianou, Reinhold Scherer and Silke Paulmann. 5.2. Developed by: Constantina Maltezou-Papastilianou. 5.3. Licensed under the wider project license: CC BY 4.0 International License.

Custom Packages

Machine Learning

`random_forest_module.py`

The purpose of this module is to use a Random Forest (RF) model to validate its reliability with a leave-one-speaker-out cross-validation approach, and to estimate Gini feature importance scores.

`Custom_packages.machine_learning.random_forest_module.calculate_gini_feature_importance`
(features: DataFrame, target: Series, model: RandomForestClassifier, group_name: str) → None

Estimates Gini feature importances and visualises them.

Parameters:

- **features** (-) – Independent variables.
- **target** (-) – Dependent variable.
- **model** (-) – Trained random forest model.
- **group_name** (-) – Name of the group for labeling outputs.

Returns: None

`Custom_packages.machine_learning.random_forest_module.perform_RF_model_assessment`
(features: DataFrame, target: Series, speaker_ids: array, group_name="") → None

Performs a leave-one-speaker-out cross-validated Random Forest (RF) evaluation. Calls relevant helper functions to print metrics (accuracy, precision, recall, F1 score) and calls relevant functions to plot confusion matrices, ROC curves and Gini feature importance scores.

Parameters:

- **features** (-) – Independent variables.
- **target** (-) – Dependent variable.
- **speaker_ids** (-) – Speaker IDs for cross-validation.
- **group_name** (-) – Name of the demographic group for labeling outputs.

Returns: None

`regression_ML_module.py`

The purpose of this module is to conduct classification and/or predictions using regression models with the machine learning package (sklearn from scikit-learn).

`Custom_packages.machine_learning.regression_ML_module.perform_LR_model_assessment`
(features: DataFrame, target: Series, speaker_ids: array, group_name="") → None

Performs a leave-one-speaker-out cross-validated (LOSO CV) Logistic Regression (LR) evaluation (scikit-learn package). Calls relevant helper functions to print metrics (accuracy, precision, recall, F1 score) and plot confusion matrices and ROC curves.

Parameters:

- **features** (-) – Independent variables.
- **target** (-) – Dependent variable.
- **speaker_ids** (-) – Speaker IDs for cross-validation.
- **group_name** (-) – Name of the demographic group for labeling outputs.

Returns: None

Stats

normality_tests_module.py

The purpose of this module is to test the normality of a dataset (df). - The larger the test statistic value is, the larger usually the deviation from a normal distribution. - A non-significant p-value suggests that the dataset appears Gaussian / normally distributed. - A significant p-value suggests that the dataset is considered skewed, hence a different distribution should be considered during analyses.

`Custom_packages.stats.normality_tests_module.shapiro_wilks_normality(df: DataFrame, variable_names: list[str], factor_names: list[str] = []) → None`

Conducts the Shapiro-Wilk normality test for specified variables and prints the results.

Parameters:

- **df** (*pd.DataFrame*) – DataFrame containing the variables to test.
- **variable_names** (*list[str]*) – Column names of variables to test for normality (e.g. acoustic features).
- **factor_names** (*list[str], optional*) – Columns for grouping data into subsets (e.g. speaker demographics).

Returns: None

Raises: **ValueError** – If DataFrame or variable_names are None or empty.

descriptives_module.py

This module is meant to calculate and return the descriptive statistics for speaker demographics and acoustic features.

`Custom_packages.stats.descriptives_module.get_acoustics_descriptives(df: DataFrame, strata_columns: list, acoustic_columns: list) → DataFrame`

Calculate descriptive statistics for each acoustic feature per stratified speaker group.

Parameters:

- **df** (*pd.DataFrame*) – The dataframe containing the dataset with acoustic features.
- **strata_columns** (*list*) – List of speaker columns to group by for stratification.
- **acoustic_columns** (*list*) – List of acoustic feature columns to calculate statistics for.

Returns: A DataFrame with calculated descriptive statistics (mean, std) for each speaker group.

Return type: `pd.DataFrame`

`Custom_packages.stats.descriptives_module.get_demographic_descriptives(df: DataFrame)`
→ dict

Calculates various descriptive statistics for speaker demographics.

Parameters: `df` (*pd.DataFrame*) – The dataframe containing the speaker demographics data.

Returns: A dictionary containing value counts and groupings for different demographic categories.

Return type: dict

regression_stats_module.py

The purpose of this module is to conduct classification and/or predictions using regression models with the statistical package (statsmodels).

`Custom_packages.stats.regression_stats_module.calculate_logistic_regression(features: DataFrame, target: Series, group_name="")` → None

Performs logistic regression (statsmodels package) on the provided dataset and exports the results to a DOCX file.

Parameters:

- **features** (*pd.DataFrame*) – Feature columns in the dataset.
- **target** (*pd.Series*) – The dependent variable (target) for the regression.
- **group_name** (*str, optional*) – A name for the demographic group, used in file naming and reports.

Returns: None

Example

```
calculate_logistic_regression(features, target, group_name="Younger")
```

`Custom_packages.stats.regression_stats_module.export_LR_summary_table_to_MS_Word(summary_table_compact: DataFrame, group_name: str)` → None

Exports the logistic regression summary table to a DOCX file.

Parameters:

- **summary_table_compact** (*pd.DataFrame*) – The compact summary table to export.
- **group_name** (*str*) – The group name used for the file name.

Returns: None

`Custom_packages.stats.regression_stats_module.format_LR_summary_table(summary_table_compact: DataFrame)` → str

Formats the logistic regression summary table for display, highlighting significant results.

Parameters: **summary_table_compact** (*pd.DataFrame*) – The logistic regression summary table.

Returns: The formatted summary table.

Return type: str

Preprocessing

feature_encoding_module.py

This module encodes categorical variables of interest into numerical values and stores them as new columns in the provided dataframe.

`Custom_packages.preprocessing.feature_encoding_module.categorical_to_numerical(df: DataFrame, categorical_variable_columns: list[str]) → DataFrame`

Converts categorical variables into numerical values and stores them in new columns within the DataFrame.

Parameters:

- **df** (*pd.DataFrame*) – The input DataFrame containing categorical data.
- **categorical_variable_columns** (*List[str]*) – A list of column names to encode as numerical values.

Returns: The updated DataFrame with encoded columns.

Return type: `pd.DataFrame`

Raises: **ValueError** – If the DataFrame or the list of categorical columns are None or empty.

Example

```
df_encoded = categorical_to_numerical(df, ["Speaker_Ethnicity", "Speaker_AgeGroup"])
```

dataset_stratification_module.py

The purpose of this module is to stratify an existing dataset based on specific criteria.

`Custom_packages.preprocessing.dataset_stratification_module.create_strata(df: DataFrame, criteria: list[str]) → dict`

Splits the dataset into smaller strata based on the provided criteria columns (e.g., ethnicity, age-group).

Parameters:

- **df** (*pd.DataFrame*) – The DataFrame to be stratified, containing the columns that are to be used as criteria.
- **criteria** (*list[str]*) – A list of column names used to stratify the DataFrame.

Returns: A dictionary containing the stratified DataFrames. "All" (`pd.DataFrame`): The original DataFrame, included in the result as a key for the entire dataset. {group_value} (`pd.DataFrame`): DataFrames grouped by the unique values in the specified criteria columns.

Return type: `dict`

Raises: **ValueError** – If the DataFrame or criteria parameters are None or empty.

Misc

visual_formatting_module.py

This module provides visual formatting utilities of script outputs, such as:

- Print clearly formatted section headings for organised outputs.
- Apply color coding to messages using ANSI escape codes.
- Highlight statistical results (e.g., p-values) with significance indicators.

These functions are designed to improve the readability and presentation of data.

`Custom_packages.misc.visual_formatting_module.highlight_significant_results(p_value: float) → str`

Format a p-value with significance indicators.

Parameters: **p_value** (*float*) – The p-value to evaluate.

Returns: The p-value rounded to three decimals: Green with ******* if $p\text{-value} \leq 0.001$. Green with ****** if $p\text{-value} \leq 0.05$. Plain text otherwise.

Return type: str

`Custom_packages.misc.visual_formatting_module.print_coloured(message: str, colour: str) → None`

Prints a message with a specified colour.

Parameters:

- **message** (str) – The message to be printed.
- **color** (str) – The colour in which to print the message. Must be one of the keys in `colours_dict`.

Returns: None

`Custom_packages.misc.visual_formatting_module.print_section_heading(heading: str) → None`

Prints a formatted section heading to visually separate script sections.

Parameters: **heading** (str) – The text to display as the section heading.

Returns: None

Example

```
>>> print_section_heading("Data Pre-processing")
=====
Data Pre-processing
=====
```

`data_visualisation_module.py`

The purpose of this module is to create a variety of plots (e.g. confusion matrices, roc curves, etc).

`Custom_packages.misc.data_visualisation_module.plot_confusion_matrix(y_true: Series, y_pred: ndarray, heading: str, model: str) → None`

Plots a confusion matrix as a heatmap using seaborn.

Parameters:

- **y_true** (pd.Series) – True labels for the confusion matrix.
- **y_pred** (np.ndarray) – Predicted labels for the confusion matrix.
- **heading** (str) – The title for the plot.
- **model** (str) – Model type (e.g. LR or RF), which determines the heatmap colour scheme.

Returns: None

`Custom_packages.misc.data_visualisation_module.plot_feature_importances(feature_names: list[str], importance_scores: array, importance_method_name="speaker_group") → None`

Plots the importance scores of features using horizontal bar plots.

Parameters:

- **feature_names** (-) – List of feature column names.
- **importance_scores** (-) – Numpy array of importance scores.
- **importance_method_name** (-) – Name of the importance scores method used, for display purposes.
- **speaker_group** (-) – Demographic group name, for display purposes.

`Custom_packages.misc.data_visualisation_module.plot_roc_curve` (`target: Series`, `y_pred_proba_cv: array`, `group_name: str`, `model: str`) → None
Plots the ROC curve for cross-validated model predictions.

Parameters:

- **target** (-) – True target values.
- **y_pred_proba_cv** (-) – Predicted probability scores, used to compute ROC-AUC metrics.
- **group_name** (-) – Name of the group for labeling outputs.
- **model** (-) – Model type (e.g. LR or RF), which determines the heatmap colour scheme.

Returns: None

helpers_module.py

This module includes common utilities used across different ML and stats scripts, such as data preparation and validation, task routing (i.e. directing to appropriate functions) and model evaluation metrics. These utilities are designed to support larger workflows and are not meant for standalone use.

`Custom_packages.misc.helpers_module.calculate_metrics` (`target: Series`, `y_pred_cv: array`, `group_name: str`) → None
Calculates and prints accuracy, precision, recall, and F1 score for cross-validated (CV) model.

Parameters:

- **target** (-) – True target values.
- **y_pred_cv** (-) – Predicted target values from cross-validation.
- **group_name** (-) – Name of the group for labeling outputs.

Returns: None

`Custom_packages.misc.helpers_module.stratified_model_evaluation_helper` (`strata_dict: dict`, `variables_dict: dict`, `analysis_func: callable`, `analysis_label: str`) → None
Prepares the data and calls the relevant functions for model cross-validation analyses. It also prints the results for each speaker demographic strata. Specifically, it focuses on ethnicity and age-group strata.

Parameters:

- **strata_dict** (*dict*) – A dictionary where the key is the strata label (e.g., speaker ethnicity or age group) and the value is the corresponding DataFrame.
- **variables_dict** (*dict*) – A dictionary where the first key is named “predictors” and holds the predictor variables (e.g., acoustic features), and the second key is named “target” and holds a single target variable.
- **analysis_func** (*callable*) – A function that performs the analysis (e.g., model evaluation).
- **analysis_label** (*str*) – A label to identify the type of analysis (e.g., “Logistic Regression Analysis”).

Returns: None

`Custom_packages.misc.helpers_module.validate_non_empty_and_exists` (`param`, `param_name='parameter'`) → None
Ensures the parameter exists and is not empty (for strings, lists, or DataFrames).

Parameters:

- **param** – The input to be validated; can be any type.
- **param_name** (*str*) – The name of the parameter, used in error messages.

Returns: None

Raises: **ValueError** – If the parameter does not exist or is empty.

License

This project (including its Python codebase) is licensed under the [CC BY 4.0 International License](#).

Index

C

[calculate_gini_feature_importance\(\)](#) (in module [Custom_packages.machine_learning.random_forest_module](#))

[calculate_logistic_regression\(\)](#) (in module [Custom_packages.stats.regression_stats_module](#))

[calculate_metrics\(\)](#) (in module [Custom_packages.misc.helpers_module](#))

[categorical_to_numerical\(\)](#) (in module [Custom_packages.preprocessing.feature_encoding_module](#))

[create_strata\(\)](#) (in module [Custom_packages.preprocessing.dataset_stratification_module](#))

[Custom_packages.machine_learning.random_forest_module](#)
module

[Custom_packages.machine_learning.regression_ML_module](#)
module

[Custom_packages.misc.data_visualisation_module](#)
module

[Custom_packages.misc.helpers_module](#)
module

[Custom_packages.misc.visual_formatting_module](#)
module

[Custom_packages.preprocessing.dataset_stratification_module](#)
module

[Custom_packages.preprocessing.feature_encoding_module](#)
module

[Custom_packages.stats.descriptives_module](#)
module

[Custom_packages.stats.normality_tests_module](#)
module

[Custom_packages.stats.regression_stats_module](#)
module

E

[export_LR_summary_table_to_MS_Word\(\)](#) (in module [Custom_packages.stats.regression_stats_module](#))

F

[format_LR_summary_table\(\)](#) (in module [Custom_packages.stats.regression_stats_module](#))

G

[get_acoustics_descriptives\(\)](#) (in module [Custom_packages.stats.descriptives_module](#))

[get_demographic_descriptives\(\)](#) (in module [Custom_packages.stats.descriptives_module](#))

H

[highlight_significant_results\(\)](#) (in module [Custom_packages.misc.visual_formatting_module](#))

M

module

[Custom_packages.machine_learning.random_forest_module](#)

[Custom_packages.machine_learning.regression_ML_module](#)

[Custom_packages.misc.data_visualisation_module](#)

[Custom_packages.misc.helpers_module](#)

[Custom_packages.misc.visual_formatting_module](#)

[Custom_packages.preprocessing.dataset_stratification_module](#)

[Custom_packages.preprocessing.feature_encoding_module](#)

[Custom_packages.stats.descriptives_module](#)

[Custom_packages.stats.normality_tests_module](#)

[Custom_packages.stats.regression_stats_module](#)

[Scripts.main](#)

P

[perform_LR_model_assessment\(\)](#) (in module [Custom_packages.machine_learning.regression_ML_module](#))

[perform_RF_model_assessment\(\)](#) (in module [Custom_packages.machine_learning.random_forest_module](#))

[plot_confusion_matrix\(\)](#) (in module [Custom_packages.misc.data_visualisation_module](#))

[plot_feature_importances\(\)](#) (in module [Custom_packages.misc.data_visualisation_module](#))

[plot_roc_curve\(\)](#) (in module [Custom_packages.misc.data_visualisation_module](#))

[print_coloured\(\)](#) (in module [Custom_packages.misc.visual_formatting_module](#))

[print_section_heading\(\)](#) (in module [Custom_packages.misc.visual_formatting_module](#))

S

[Scripts.main](#)
module

[shapiro_wilks_normality\(\)](#) (in module [Custom_packages.stats.normality_tests_module](#))

[stratified_model_evaluation_helper\(\)](#) (in module [Custom_packages.misc.helpers_module](#))

V

`validate_non_empty_and_exists()` (in module
Custom_packages.misc.helpers_module)

Python Module Index

c

Custom_packages

Custom_packages.machine_learning.random_forest_module

Custom_packages.machine_learning.regression_ML_module

Custom_packages.misc.data_visualisation_module

Custom_packages.misc.helpers_module

Custom_packages.misc.visual_formatting_module

Custom_packages.preprocessing.dataset_stratification_module

Custom_packages.preprocessing.feature_encoding_module

Custom_packages.stats.descriptives_module

Custom_packages.stats.normality_tests_module

Custom_packages.stats.regression_stats_module

s

Scripts

Scripts.main